TrainAudit: Silent Error Detection for LLM Training via Verified Guarded Relational Constraints

Qingsong Cai*
Sichuan University
Chengdu, China
qingsong1010@gmail.com

Shikai Li*
West China Hospital, Sichuan University
Chengdu, China
lishikai@wchscu.edu.cn

Zhengmao Ye
Sichuan University
Chengdu, China
yezhengmaolove@gmail.com

Hao Tang
Sichuan University
Chengdu, China
tangguan250@gmail.com

Mingjie Tang
Sichuan University
Chengdu, China
tangrock@gmail.com

Ran Tao
iQuest
Beijing, China
rtao02@ubiquant.com

Bryan Dai
iQuest
Beijing, China
cbdai@ubiquant.com

ABSTRACT

Large language model training is susceptible to *silent errors*: faults that violate what the training algorithm prescribes—not what the runtime enforces—yet produce no exceptions or anomalous metrics. Such errors go undetected by standard monitoring, silently degrading model quality and wasting tens of thousands of GPU-hours. The root cause is a gap between what current monitors observe and what correctness requires: scalar metrics discard the rank-level, topology-level, and phase-level relations that silent errors depend on, while raw tracing materializes millions of records without saying which combinations are illegal—the missing primitive is *integrity constraints* over the training trace. We present TrainAudit, which uses LLM agents to derive and validate framework-aware integrity constraints from source code and documentation, then enforces them over runtime traces to detect violations of distributed-training semantics. On **Real-SE**, a benchmark of real silent errors, TrainAudit detects **17/18 (94.4%)** buggy instances with **0/17** fixed-side false positives; TrainCheck detects **5/17 (29.4%)** on the completed case-aligned surrogate runs. Deployed on a presumed-clean 256×H200 production job, TrainAudit fired *before the first optimizer step*, preventing ∼43,000 GPU-hours of silent corruption; the fault it exposed was **one previously unrecorded bug**, since confirmed and fixed upstream.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Software and its engineering** → **Software reliability**; *Software testing and debugging*.

KEYWORDS

distributed training, large language models, silent errors, data quality, integrity constraints, constraint mining

PVLDB Reference Format:

Qingsong Cai, Shikai Li, Zhengmao Ye, Hao Tang, Mingjie Tang, Ran Tao, and Bryan Dai. TrainAudit: Silent Error Detection for LLM Training. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Training a large AI model is a long-running distributed state transition. At each step, thousands of GPUs compute partial gradients, exchange tensors through collective communication, update optimizer state, restore or write checkpoints, and advance framework counters under a specific parallel topology. At frontier scale, a single run occupies thousands of GPUs for weeks [11]; an undetected silent error corrupts the trajectory for the remainder, wasting tens of thousands of GPU-hours before the damage—if ever—surfaces.

A *silent error*[1] occurs when this transition deviates from the intended training algorithm without becoming obviously invalid: one rank may skip a synchronization, overwrite an accumulated gradient, restore only part of a checkpoint, or advance an optimizer counter at the wrong phase—yet the program does not crash, tensor shapes and dtypes remain legal, and the loss curve may still look plausible. Such errors are *semantic* in nature—they violate what the training algorithm prescribes, not what the runtime enforces—yet they are *silent* in manifestation, because existing runtime infrastructure checks only structural validity (types, shapes, exceptions) and has no interface for algorithmic correctness. Detecting them therefore requires recovering the semantic correctness conditions that current monitoring leaves unchecked.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

*Qingsong Cai and Shikai Li contributed equally to this work.

[1]Following prior work [9, 10], we use *silent errors* to denote software defects that silently corrupt training state without raising exceptions—distinct from hardware-induced silent data corruption (SDC) [2, 7], which is orthogonal and addressed by ECC.

Existing detectors observe this transition through limited oracle sources. Metric-based monitors are easy to deploy, but they watch downstream symptoms such as loss spikes, NaNs/Infs, gradient-norm outliers, and throughput anomalies. Healthy-trace approaches can learn useful execution regularities [10], but regularity in previous traces is not the same as a framework-level correctness obligation under a new topology or phase. Reference-based checking obtains a stronger oracle by comparing intermediate tensors against a trusted or alignable implementation [9], but such a reference is often unavailable for production-scale runs. Plan verification can reason about whether a distributed execution plan matches a logical specification [14], but it does not audit the dynamic state transitions that occur as counters, checkpoint state, optimizer metadata, and communication groups evolve during training. Our insight is that the hard silent errors are not signal-free; their signal is stranded below current monitoring interfaces.

Figure 1 shows a DeepSpeed ZeRO-2 bug of this form. An off-by-one error in the `micro_step_id` counter resets a gradient buffer when it should be accumulated. The run continues, the tensors have legal shapes and dtypes, and the loss curve remains nearly unchanged. Yet the live execution already contains evidence of the error: the current micro-step, the buffer operation, the accumulation phase, and the optimizer boundary. None of these fields is a reliable alarm by itself, yet together they already carry the signal—the evidence of the error is present in the runtime state, but in a form no existing monitor checks. Concretely, they define a relation that correct training must satisfy: in a given accumulation phase, the buffer operation must match the intended update rule.

This example illustrates one side of the data-management problem behind silent-error detection: scalar monitoring observes too little—loss, gradient norm, NaN/Inf counters, and throughput discard rank-level, object-level, topology-level, and phase-level relations that the signal depends on. The naive remedy—recording everything—creates the complementary problem. Raw tracing can materialize millions of records about tensor states, optimizer counters, communication events, checkpoint state, rank groups, topology lineage, and framework metadata, yet it does not say which combinations are illegal. What is needed, then, is a way to locate the meaningful signal within the rich runtime state—observing more than scalar metrics, yet with enough guidance to avoid being lost in millions of undifferentiated records.

No explicit specification of correct training behavior exists, but we observe that framework source code and confirmed bug fixes jointly encode both what can go wrong and what must hold: each fix implicitly reveals a correctness condition that was previously unwritten. This paper starts from these latent obligations and asks whether they can be recovered systematically and operationalized as online checks, without requiring a trusted reference run. Two difficulties stand between this idea and a deployable detector: each recovered obligation is valid only under specific topology and phase conditions—enforced unconditionally, it floods the operator with false alarms on correct runs—and recovering obligations at scale from framework artifacts requires automation that is itself error-prone.

We propose TrainAudit, which addresses both by separating obligation *discovery* (offline, LLM-assisted, verified against counterexamples before deployment) from obligation *checking* (online,

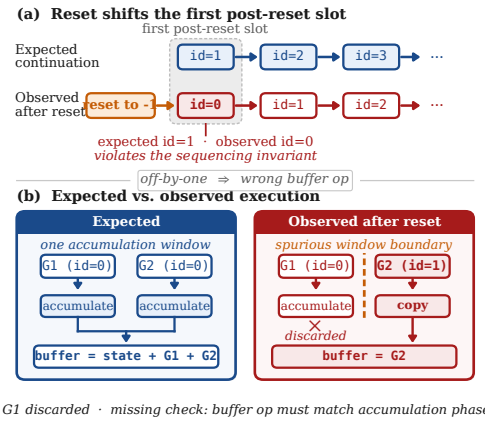


Figure 1: Silent error in a DeepSpeed ZeRO-2 run: an off-by-one `micro_step_id` causes a gradient buffer to be overwritten when it should be accumulated. The relevant facts—micro-step, gradient-buffer operation, accumulation phase, and optimizer boundary—are already present in the runtime trace; the missing object is the guarded relation that identifies the illegal combination.

fully deterministic SQL queries over a relational training trace), so that the online path contains no LLM and no unverified rule.

Our empirical study shows that this formulation captures recurring failures rather than isolated anecdotes. We analyze a **392-record upstream silent-error evidence corpus** from Megatron-LM, DeepSpeed, OLMo, and OLMo-core. The recurring relations we induce from it transfer to a locked held-out test: a frozen catalog snapshot matches **92.3%** of those bugs, with all three predicates groundable for **91.0%**; the catalog must nonetheless remain extensible, since new framework code keeps introducing object-specific obligations (§5.4). On Real-SE, a reproducible set of 17 buggy/fixed commit-pair replays plus 1 observability-boundary case spanning the corpus's descriptive subsystem categories, TrainAudit detects **17/18 (94.4%)** buggy instances, compared with **5/17 (29.4%)** for TrainCheck and **0/17 (0.0%)** for metric-based monitoring on the completed case-aligned surrogate runs, while producing **0/17** fixed-side false positives. Deployed on a presumed-clean 256×H200 production job, a verified rule fired before the first optimizer step, preventing ~43,000 GPU-hours of silent corruption; we reported the fault upstream, where maintainers confirmed and fixed this previously unrecorded bug.

In summary, we make the following contributions:

1) We define a *training trace data model*, a relational data model for distributed AI training executions covering tensor state, optimizer state, call events, communication/checkpoint events, rank groups, and topology lineage.

2) We formalize silent-error detection as checking *runtime-guarded integrity constraints* over live training traces, where each constraint specifies which runtime fields to read, which tuples can be compared under the current topology, when the relation is valid, and what violation predicate to execute.

Figure 2: Production silent error (Appendix H, Case 1): losses overlap for 2,000 steps while downstream quality diverges 15.7 points; the rule fires at step 1.

3) We develop a reproducible template-induction protocol and a *verified mining and execution* pipeline: a versioned relation catalog narrows candidate generation and evolves with new framework evidence, while counterexample checks, healthy-run replay, and topology-aware validation govern acceptance before deterministic SQL execution.
4) We build *Real-SE*, a reproducible benchmark of real buggy/fixed commit-pair replays and an explicit observability-boundary case, and use it to evaluate TrainAudit against trace-template and metric-based detectors (§5.2).

2 BACKGROUND AND MOTIVATION

Diagnosing a silent error today is manual and non-reusable: an engineer gathers whatever evidence a run exposes and reasons over it with the framework source, restarting for every incident. *Yet the conditions that correct training must satisfy have been in the data all along; what is missing is not the evidence but a way to mine it.* §2.1 defines silent errors and surveys prior detection paradigms by oracle source; §2.2 distills the gaps into three data-management challenges (C1–C3).

2.1 Silent Errors and Why Existing Approaches Miss Them

A *silent error* is a fault in distributed training that produces numerically valid tensors and raises no exception, yet leaves a verifiable constraint on training state violated at a known hookpoint. Root causes span software bugs (e.g., a missing all-reduce), framework misuse, and configuration drift; the common signature we exploit is no crash, surface-plausible per-step metrics, and diverging final quality. Out of scope are faults whose only manifestation is asymptotic convergence drift; they are unobservable at runtime by any method and account for the residual gap in §5. Silent errors can strike at every phase of a hybrid-parallel step: frontier training shards a job across four parallelism dimensions (DP/PP/TP/EP), and each step exposes five silent-error hotspots aligned with standard runtime hookpoints (Appendix I gives the panorama). Meanwhile, outcome metrics surface faults hundreds of steps too late: loss and gradient norm track a healthy baseline long after corruption is introduced (Figure 2).

A silent-error detector requires a *detection oracle*: a source of truth against which observed runtime state is judged correct or corrupt. Prior systems differ primarily in what they treat as that source,

Table 1: Silent-error detectors and structural limitations. Real-SE contains 18 cases for TrainAudit; baseline columns report the 17 completed case-aligned surrogate runs (O-003 pending). "–" denotes methods without a directly comparable runtime result.

Method	Mode	Oracle	Real-SE	Key limitation
Loss / NaN monitor	Online	Outcome metrics	0/17	Late, uninformative
TrainCheck	Online	API trace templates	5/17	No topology awareness
TTrace	Offline	Reference impl. diff	–	Needs alignable reference
TrainVerify	Static	Logical specification	–	Plan-only, no runtime
TrainAudit	Online	Bug-derived constraints	**17/18**	(this work)

and each choice imposes a structural ceiling: outcome-metric monitors fire only after the macro curve itself bends, because the curve averages a fault's fine-grained signature away; behavioral trace templates learn from the healthy-run trace of a single process and cannot express relations between ranks; differential testing needs a trusted, alignable reference implementation and runs offline; and plan verification covers only what is decidable before the first step executes. Table 1 consolidates the four paradigms and reports their Real-SE-aligned results under the protocol of §5.1: metric-based monitoring catches **0/17** completed surrogate cases, while TrainCheck, the strongest trace-template detector, catches **5/17**.

TrainAudit occupies the unexplored cell of this design space: it derives deployable checks from *historical bug fixes*, attaining cross-rank coverage, freedom from a trusted reference, and online execution at once. This position is feasible because of a structural property established in §3—silent errors repeatedly violate reusable relations over runtime training facts, even as the concrete template catalog grows with framework code. Turning those relations into trustworthy online checks requires overcoming three obstacles (§2.2).

2.2 Challenges

C1: A constraint is valid only under a guard. Whether a relation over training state must hold is itself a conditional integrity constraint: it holds on some tuples and not others. Within a single tensor-parallel run, `layernorm.weight` is replicated across ranks and must be bitwise equal, whereas `query_key_value.weight` is partitioned and must differ—yet a single cross-rank equality check matches both. Enforced unconditionally, it flags every legitimately partitioned tensor, burying real faults under false alarms; a constraint carrying no guard for tensors, topology, and phase is unusable in production (§3.2).

C2: Correctness constraints are latent and must be mined, not assumed. The constraints that define correct training are never written down as such: they are latent in the framework's implementation and the algorithm's mathematics, and only their *violations* ever surface—as a fixed bug or a documented caveat. This rules out profiling them from a data instance the way integrity constraints

usually are: a single clean run exhibits countless regularities, but a regularity that merely happened to hold is not one that must hold, and enforcing it fires false alarms on the next run. The authoritative source is the code and algorithm that generate the data, not the data itself—yet reading a constraint off that source is error-prone, because what the documentation states and what the implementation enforces routinely diverge. A candidate that looks correct on paper is quietly violated by the framework's own initialization path; without validation grounding each candidate in the actual code, a mined library is itself a source of false positives (§4.3).

C3: Checking must keep pace with training. The check runs online, on the critical path of every training step: a detector that slows the step, or falls behind it, will be turned off. Yet the state worth checking is large—parameters, gradients, and optimizer state across every rank—and inspecting it in full at every step is too expensive to sustain in production. The challenge is to keep the check cheap enough to run continuously without stealing time from training, yet thorough enough to still catch real faults (§4.4, §4.5).

These challenges structure the paper. For C1, §3.2 decomposes every deployable constraint into three predicates—measurement schema, agreement topology, validity precondition—whose guards the Invariant Miner instantiates per framework (§4.3). For C2, the Miner treats LLM-proposed candidates as untrusted, admitting a constraint only after counterexample verification, healthy-run replay, and topology-aware scope validation (§4.3). For C3, the Data Collector captures only fields that accepted constraints read (§4.4) and the Verifier prunes rules inapplicable under the active topology (§4.5); overhead is quantified in §5.5. §3 first establishes the empirical foundation: real silent errors reuse guarded relations over runtime training facts, while the catalog of concrete templates evolves with the frameworks.

3 EMPIRICAL STUDY AND PROBLEM FORMULATION

Detecting a silent error is an integrity-checking problem missing its constraints: every training step records what actually happened, but nothing in the data says what should have happened. This correctness standard is not part of the training state; it belongs to the process that produces the data—the training code and the algorithm. We develop this in three steps. §3.1 studies the 392-record upstream evidence corpus and finds that recognition requires an external correctness relation. §3.2 then formalizes how such a relation is checked—aligned back onto the data as a guarded relational query—an alignment that reduces to three predicates: a measurement schema, an agreement topology, and a validity precondition. §3.3 establishes formal properties of this decomposition—in particular a schema-relative observability ceiling that §5 measures directly.

3.1 Empirical Study: The Correctness Standard Is External

Corpus construction. We establish §3's premise—that the standard of correctness lives *outside* the training data—on a curated corpus of **392 upstream silent-error records** from Megatron-LM, DeepSpeed, OLMo, and OLMo-core. The released corpus combines a commit-backed collection with an earlier issue/PR-curated collection, deduplicated by upstream incident (Megatron-LM 110, DeepSpeed 123, OLMo 77, OLMo-core 82). Commit diffs and messages are the preferred evidence because they identify the changed training path and a concrete before/after semantics. Issue text is supporting evidence rather than a sufficient detector-benchmark source: reports often omit the exact buggy revision, trigger configuration, or executable oracle, and sometimes conflate multiple symptoms.

Table 2: Descriptive distribution of the 392-record evidence corpus by one coarse primary subsystem label per record (coding methodology in Appendix B). The labels summarize the corpus rather than define an exhaustive fault ontology. *Parallelism* gives the typical applicable dimension ("any" = topology-independent); *Loop Stage* gives the stage at which the coded failure first becomes observable.

Primary label	Records	Parallelism	Loop Stage
Numerical	75	any	Fwd/Bwd
Control Flow	50	any	All
Grad Sync	46	DP	Backward
Checkpoint	32	DP/TP/PP	Save/Load
MoE	28	EP	Fwd/Bwd
Optim State	28	DP/FSDP	Optim Step
Data Loading	28	DP	Pre-step
Communication	26	DP/TP/PP/EP	All
Dtype	24	any	Fwd/Bwd
Loss Comp.	23	any	Forward
Sharding	13	TP/PP	Init/Fwd
LR Schedule	13	any	Optim Step
Offload	6	DP/FSDP	Optim Step
Total	**392**		

A candidate is retained when the available evidence supports a semantic correctness failure in a training-critical path rather than documentation, performance, or refactoring, and describes silent continuation as the primary behavior rather than an immediate exception. We require enough information to extract the affected semantic objects and intended relation, but do not treat this screening decision as physical reproduction: only the runnable subsets verify buggy and fixed behavior directly. We collapse duplicate reports of the same incident. To summarize where the retained incidents occur, we assign each record one coarse primary subsystem label using an inductive coding process and report the distribution in Table 2. These labels are descriptive bins for corpus reporting, not a claim that silent errors form a unique or exhaustive ontology, and they are not the relation-template space used by TrainAudit. Appendix B reports provenance strength separately: 289 records carry a fixed-commit identifier, 96 are issue/PR-only records, and 7 retain only a curated source summary in the merged artifact. The corpus is therefore heterogeneous evidence for descriptive coding and catalog induction, not 392 equally reproduced bugs or an exhaustive all-commit census over a preserved date window.

Finding 1: Recognition requires an external correctness relation. The retained evidence does not identify a fault from an unlabelled runtime record alone; it first supplies an intended relation from the training code, algorithm, or confirmed fix. Runtime state may then contain a witness—for example unequal replicas or an incorrect call count—but it does not encode why those values should agree or how many calls are correct. In the physically replayed subset, ordinary operator-facing signals such as loss and gradient norms can remain plausible despite the violation. We do not claim that every one of the 392 records has a bitwise-identical healthy/buggy trace; the corpus supports the narrower structural claim that recognition needs an external standard of correctness.

Finding 2: Silent errors recur at the level of runtime relations. Across the coded records, faults from different frameworks repeatedly instantiate the same kinds of obligations—replicated state should agree, scaled quantities should carry the intended factor, producers should precede consumers, and saved state should be restored completely. Yet these relations apply to different semantic objects and under different topology and phase conditions. This motivates an intermediate representation between the coarse corpus labels and framework-specific checks: a catalog of reusable relation templates whose reproducibility and coverage we evaluate in §5.4.

Finding 3: Whether a constraint must hold depends on topology and phase. The two rightmost columns of Table 2 summarize the guard structure visible even at this coarse coding level. Several labels are tied to specific parallelism dimensions—Grad Sync to DP, Sharding to TP/PP, and MoE to EP—and failures first become observable at different points of the training loop. A deployable check must therefore carry with it the topology under which its relation is required and the phase at which it is valid. §3.2 formalizes exactly this requirement as a three-predicate decomposition.

3.2 Problem Formulation and Three-Predicate Decomposition

Building on §3.1's finding that the correctness constraint lives outside the data, this subsection formalizes what it means to check such a constraint at runtime and reduces every deployable check to three predicates. We model a training run as a sequence of states, cast a correctness constraint as a *semantic invariant* over them, and decompose each invariant into *what to measure*, *which ranks must agree*, and *when the check applies*. Each state contains the objects that a silent error can corrupt: model parameters on each rank, gradients, optimizer state, training-loop control variables, framework metadata such as `tensor_model_parallel`, and the active parallel topology τ. A silent error occurs when the actual next state differs from the intended next state, while the program raises no exception.

Definition 3.1 (Silent Error). A *silent error* at step t is a state $\tilde{s}_{t+1} = f_{\text{actual}}(s_t)$ such that $\tilde{s}_{t+1} \neq f_{\text{ideal}}(s_t)$ and no runtime exception is raised.

Since f_{ideal} is unavailable at runtime, TrainAudit instead checks *semantic invariants*: relations that must hold in every correct training state—replicated parameters should agree across the right ranks, counters should advance with the training loop, and framework metadata should be consistent with the active topology. Crucially, each training state is already materialized as relational trace tuples, so such an invariant becomes a *guarded relational constraint* that TrainAudit evaluates as a query over the trace, where a violation surfaces as a non-empty result. The remaining difficulty is therefore to make each constraint precise enough to deploy, which is what the three-predicate decomposition below provides.

Definition 3.2 (Semantic Invariant). A *semantic invariant* is a predicate $p : \mathcal{S} \to \{0, 1\}$ such that $\forall s_0, t : p(f^t_{\text{ideal}}(s_0)) = 1$. A violation $p(\tilde{s}) = 0$ is sound evidence of corruption.

Consider a DP=2 run where a missing `allreduce` causes rank 0 and rank 1 to accumulate different gradient updates silently. In state terms, $\theta_{\text{rank0}} \neq \theta_{\text{rank1}}$ while no exception is raised—a silent error by Definition 3.1. The natural invariant is $p(s) = [\theta_{\text{rank0}} = \theta_{\text{rank1}}]$, which is violated (Definition 3.2). But three questions must be answered to make it useful in production: *what exactly to compare?* (parameter checksums, not raw tensors), *on which parameters?* (DP-replicated weights only—TP-sharded `query_key_value` weights are *supposed* to differ), and *when?* (after step-1 synchronization, not at initialization). These three questions map directly to the three-predicate decomposition below.

Every deployable constraint must answer these three questions, so we decompose each constraint into three predicates:

$$p(s) \;=\; \pi_{\text{schema}}(s) \;\wedge\; \pi_{\text{topo}}(s;\, \tau) \;\wedge\; \pi_{\text{precond}}(s;\, t). \qquad (1)$$

π_{schema} selects the fields of s to read, such as parameter checksums, gradient norms, or function-call counts. π_{topo} encodes which ranks or objects should agree under the active topology τ. π_{precond} guards the check by training step and framework metadata, preventing valid states such as intentionally sharded tensors from being reported as violations. The three predicates are jointly necessary: dropping any one leaves the others without production usability—a bare cross-rank comparison misfires on correctly-sharded tensors, and an unguarded equality check fires before step-1 synchronization—which our drop-one ablation quantifies (§5.3). This decomposition is the formal answer to C1: the guards are part of the constraint, not side information attached after the fact.

3.3 Formal Properties of Guarded Constraints

We briefly formalize the three-predicate decomposition as a Boolean-algebra fragment over the trace schema, establishing three properties that §5 measures directly. Write $G_c = \pi_{\text{topo}} \wedge \pi_{\text{precond}}$ for the *guard* of constraint c. The *violation set* on trace D is

$$V_c(D) \;=\; \left\{\, tup \in D : G_c(tup) \;\wedge\; \neg\, \pi_{\text{schema}}(tup) \,\right\}, \qquad (2)$$

and c fires on D iff $V_c(D) \neq \emptyset$. For relational checks, π_{schema} is evaluated per *comparison group*—the guarded tuples sharing the constraint's grouping key, e.g., all replicas of one parameter at one step—and $V_c(D)$ contains the tuples of every violating group (single-tuple checks are singletons); this is exactly the `WHERE/GROUP BY/HAVING` shape the Verifier compiles to (§4.5).

Proposition 3.3 (Soundness). *If p_c is a semantic invariant (Definition 3.2) and c fires on the trace $\tilde{D}_t$ of an actual execution, then $\tilde{s}_t$ deviates from the ideal trajectory.*

Immediate: firing implies $p_c(\tilde{s}_t) = 0$, which by Definition 3.2 places $\tilde{s}_t$ off the ideal trajectory. The guarantee thus holds only for *verified* invariants that pass Eq. (3), not arbitrary LLM-proposed candidates.

Proposition 3.4 (Guard Monotonicity). *If c_1, c_2 share the same schema condition and $G_1 \Rightarrow G_2$, then $V_{c_1}(D) \subseteq V_{c_2}(D)$ for every trace D.*

Immediate from the subset relation on the guard conjuncts. Dropping any conjunct from a guard can therefore only weakly increase the false-positive count on a fixed clean corpus—a set-theoretic fact about *any* guarded predicate, not a property specific to our templates. §5.3 confirms the direction and measures the magnitude of each guard's effect.

The two propositions above are elementary once the decomposition is stated. The result below characterizes the schema-relative limit without assuming that every numerical difference is itself a sound, reference-free oracle.

Theorem 3.5 (Schema-Relative Observability Ceiling). *Let $\mathrm{proj}_\Sigma(D)$ denote a trace projected onto schema Σ, and let $\mathcal{L}_\Sigma$ be the set of such projections admitted by correct executions under the applicable topology and phase guards. If $\mathrm{proj}_\Sigma(\tilde{D}) \in \mathcal{L}_\Sigma$ for a faulty trace $\tilde{D}$, no sound guarded predicate over Σ can distinguish $\tilde{D}$ from all correct executions. Conversely, if the faulty projection lies outside $\mathcal{L}_\Sigma$ and the predicate language can express a separator, then a guarded predicate over Σ can detect it.*

Proof. If $\mathrm{proj}_\Sigma(\tilde{D}) \in \mathcal{L}_\Sigma$, some correct execution has the same observable projection. Any predicate over Σ has the same value on the two projections, so a predicate that fires on $\tilde{D}$ also fires on a correct execution and is not sound. For the converse, an expressible separator is a schema predicate that excludes the faulty projection while accepting $\mathcal{L}_\Sigma$; conjoining it with the applicable topology and phase guards yields the required constraint. □

The bound is tight in practice: O-003's sub-percent z-loss change is present numerically, but remains within the values admitted by clean executions and has no sound reference-free separator under the current schema. The survey case O-022 similarly requires detecting a statistical dataset bias from a source-level record without a reproducible commit pair. Both therefore remain observationally indistinguishable from some correct execution under the available online information (§5.2). This ceiling is a fundamental limit of what runtime checking can observe, not a limitation of the framework.

How TrainAudit mines such three-predicate constraints from framework artifacts (C2) and executes them at training pace (C3) is the subject of §4.

4 TRAINAUDIT: MINING AND CHECKING VERIFIED CONSTRAINTS

This section describes how TrainAudit addresses C1–C3. §4.1 presents the architecture and the offline/online separation that keeps runtime checking deterministic. The remaining subsections follow one pipeline: the Pattern Catalog distills recurring corpus relations into reusable templates (§4.2); the Invariant Miner scopes each template into a verified rule (§4.3); the Data Collector records the fields those rules read (§4.4); and the Verifier evaluates them over the live trace (§4.5). One requirement runs through every stage: a rule must be broad enough to catch a recurring silent-error pattern yet narrow enough to stay silent on correct executions.[2]

4.1 Architecture and Running Example

TrainAudit evaluates constraints over a relational *training trace*, so that a silent error surfaces as a non-empty query result. As shown in Figure 3, three components split along an offline/online boundary and jointly address the challenges of §2.2. Facing C1 and C2, the offline **Invariant Miner** (§4.3) turns Pattern Catalog templates into verified, guarded constraints. It fills the topology and phase guards that make a relation deployable and admits a constraint into the library only after it survives counterexample verification and healthy-run replay. To address C3, the online **Data Collector** (§4.4) hooks the training loop at its canonical lifecycle points and materializes into the trace database only the fields the accepted constraints read. The online **Verifier** (§4.5) evaluates each constraint as a SQL query over that trace under topology-aware pruning and coarse-to-fine checking; when a query returns rows, it reports the violating constraint, rank, and step.

The design turns on one separation: constraint *discovery* is offline and LLM-assisted, whereas constraint *checking* is online and fully deterministic. The LLM is invoked only inside the Invariant Miner when a catalog/framework version is prepared, rather than once per job; the online path—runtime hooks, a DuckDB trace database, SQL queries, and static assertions—contains no LLM call. TrainAudit attaches as a drop-in: one context-manager wrapper registers all hooks with *no user code changes*. A thin per-framework adapter (30–150 LoC) maps abstract lifecycle points to framework entry points, so a pinned library version can serve all five supported frameworks (§5.6).

We thread one artifact through the whole system: a cross-rank replication bug in Megatron-LM's SwitchMLP router weights, where a missing synchronization leaves two replicas that should be bitwise equal silently divergent. The correctness relation is easy to state yet unsafe to deploy unguarded. Replicated parameters must share a checksum across TP ranks, but intentionally sharded parameters must not be compared at all; the bare check therefore flags 57 correctly sharded `query_key_value.weight` tensors on a clean TP = 2 run. TrainAudit scopes the cross-rank replication template before deployment—comparing checksums only when TP > 1 and the parameter is not tensor-parallel sharded—so the same check stays silent on this clean run yet returns the divergent `router.weight` in the buggy run.

4.2 Pattern Catalog: From Corpus Evidence to Reusable Rule Templates

Why a catalog is necessary. The coarse corpus labels of §3 summarize where failures occur, but they do not specify a check: "gradient synchronization", for example, does not say which states should agree, across which ranks, or at what point in the step. Writing one check per incident would preserve these accidental details and

[2] *Terminology conventions.* To avoid overloading the word "tier", the remainder of this paper uses disjoint naming conventions: *schema collection tiers* S0–S6 denote trace field sets and their cost trade-offs; *rule-trigger tiers* A0/A1 denote the software layer at which a rule fires (A0 = PyTorch public interface, A1 = framework adapter). Within the diagnosis chain (Appendix G), the labels C1/C2 name its two stages and are unrelated to the challenges C1–C3 of §2.2.

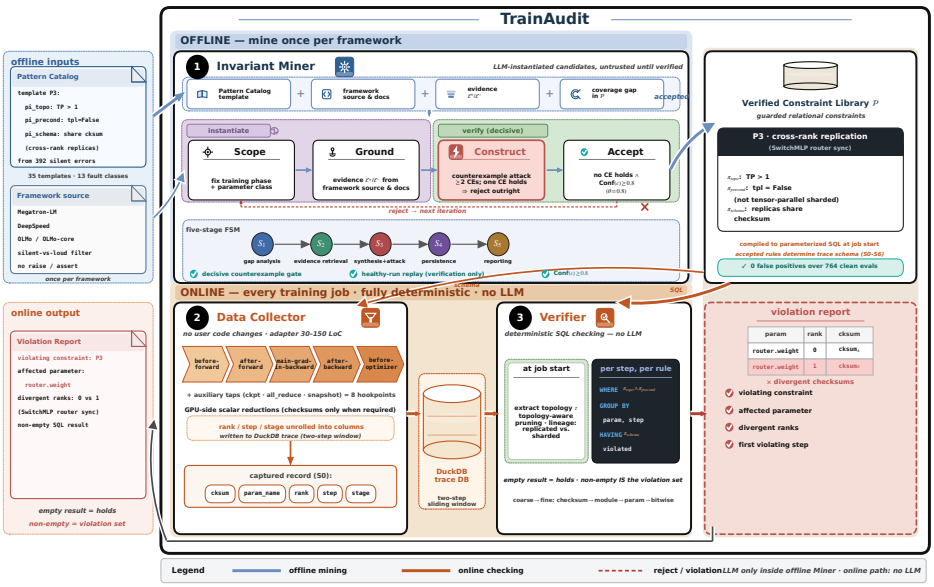


Figure 3: End-to-end workflow of TrainAudit. ❶ The Invariant Miner runs *offline* when preparing a catalog/framework version, mining and verifying guarded relational constraints from framework source code. ❷ The Data Collector and ❸ the Verifier run *online*—during every training job—to capture runtime state into a trace database and execute the accepted constraints without any LLM in the loop.

transfer poorly. At the other extreme, asking the LLM to invent a relation while inspecting each framework gives the miner an unbounded and moving search space. The Pattern Catalog bounds this search by maintaining the relation families currently supported by the evidence, while leaving framework objects and applicability conditions to later grounding.

Catalog construction and use. Each catalog entry records a framework-independent relation (e.g., equality or ordering), its semantic object roles and observable violation witness, and the topology and phase guards under which it may apply; we admit entries supported by recurring bug-and-fix evidence and version the catalog as new evidence arrives. The Invariant Miner then grounds a selected entry into framework-specific fields, topology groups, and lifecycle stages and verifies the resulting executable rule before deployment; we evaluate a frozen catalog snapshot in §5.4, with the complete construction protocol in Appendix D.

4.3 Invariant Miner: Instantiating Templates into Verified Rules

A catalog template is still not an executable rule: it names a relation worth checking, but the relation must be scoped to the code, topology, and training phase of a concrete framework. Letting an LLM fill in that scope directly is fast but unsafe—a plausible-looking candidate may omit a topology guard, apply at the wrong phase, or encode a documentation-level assumption that the real initialization path violates. The Invariant Miner therefore treats every LLM-instantiated candidate as *untrusted until verified*. It admits a candidate into the verified constraint library $\mathcal{P}$ only after counterexample verification, healthy-run replay, and topology-aware scope validation; the resulting library then executes online with no LLM in the training loop. The LLM performs bounded template instantiation—filling in $\pi_{\text{schema}} \wedge \pi_{\text{topo}} \wedge \pi_{\text{precond}}$—rather than open-ended constraint generation.

Mining one constraint follows a fixed path, and the miner applies its four stages in causal order rather than as an independent checklist. It first **scopes** the candidate to a training phase and parameter class, fixing $\pi_{\text{topo}} \wedge \pi_{\text{precond}}$; without this step, a cross-rank equality rule would also match correctly sharded parameters. Only once the scope is constrained does it **ground** π_{schema}, collecting supporting evidence $\mathcal{E}^+$ and contradicting evidence $\mathcal{E}^-$ from the call sites and module attributes retrieved from the framework source. The grounded candidate then enters the adversarial **construct** stage, which synthesizes at least two counterexamples $\mathcal{CE}$ that would make the rule fire on a clean run if its scope were still too broad. Finally, the miner **accepts** the candidate only when no counterexample validates and the confidence score clears a threshold:

$$\begin{aligned}\textsc{Accept}(c) \Leftrightarrow \big(\forall ce \in \mathcal{CE} : \textsc{Verify}(ce) \neq \texttt{HOLDS}\big) \\ \wedge\ \big(\textsc{Conf}(c) \geq \theta_{\text{conf}}\big),\end{aligned} \tag{3}$$

The two conjuncts of Eq. (3) are not symmetric. The counterexample gate is *decisive*: a single confirmed counterexample—a constructed state on which the candidate fires during a provably correct execution—shows the candidate is not a semantic invariant (Definition 3.2), and it is rejected outright; no confidence score overrides this verdict. $\textsc{Conf}(c) \in [0, 1]$ plays the complementary *screening* role: it aggregates evidence strength and counterexample-test outcomes (persisted record in Appendix E.2) on top of a hard floor of two independent evidence sources. Because the score is LLM-produced and uncalibrated, it is never treated as a correctness judgment; it only keeps thin-evidence survivors out of the library,

Algorithm 1 Counterexample-guided constraint mining loop.

Require: Pattern catalog C, framework source F, launch script L
Ensure: Verified constraint library $\mathcal{P}$

$\mathcal{P} \leftarrow \emptyset$ // initialize library
while coverage_gap$(C, \mathcal{P}) > 0$ **do**
  $pat \leftarrow S_1$.PickPattern$(C, \mathcal{P})$ // S_1: gap analysis
  $ctx \leftarrow S_2$.GatherEvidence(pat, F, L) // S_2: evidence gathering
  $c \leftarrow S_3$.Synthesize(pat, ctx) // S_3: candidate proposal
  $\mathcal{CE} \leftarrow S_3$.Attack$(c)$ // S_3: counterexample attack
  if $\left(\forall ce \in \mathcal{CE} : \textsc{Verify}(ce) \neq \texttt{HOLDS}\right) \wedge \left(\textsc{Conf}(c) \geq \theta_{\text{conf}}\right)$ **then**
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{c\}$ // S_4: persist accepted
  end if
  S_5.Report$(c, \mathcal{P})$ // S_5: feedback (accept/reject)
end while
return $\mathcal{P}$

with θ_{conf}=0.8 as a conservative default. The threshold therefore controls which counterexample-free survivors enter the library, but it never overrides a confirmed counterexample. Precision is measured separately end-to-end (0 false positives over 764 clean evaluations, §5.3).

For a selected catalog version, Algorithm 1 iterates over uncovered deployment-priority templates until the target library gap closes.

Figure 4 traces this funnel on the SwitchMLP cross-rank replication example of §4.1: the unguarded rule still flags the 57 correctly sharded `query_key_value.weight` tensors, and each added guard removes one failure mode until the accepted rule stays silent on all 57 yet still detects the `router.weight` synchronization bug. The same scope-first path applies to the other catalog templates.

We realize the mining loop as a five-stage finite-state machine with specialized LLM roles (gap analysis, evidence retrieval, synthesis, persistence, reporting; state diagram in Appendix E); S_3 is the only branching stage and hosts the verification funnel above. This decomposition is an engineering convenience for prompt modularity; the contribution is the verified mining loop, not multi-agent orchestration. Mining is an offline cost paid when onboarding a framework or updating the pinned catalog/framework version. Unchanged templates and adapters are reused, while affected scopes are re-grounded and verified (cost quantified in §5.5). The miner and runtime pruner divide responsibility for scope enforcement: the funnel writes the tightest scope supported by static evidence, and at job start the pruner reads the live τ and disables rules whose preconditions do not match the live configuration. For cross-rank replication, the miner leaves the rule with π_{topo} = `[TP>1]` and π_{precond} = `[tpl=False]`, where `tpl` marks intentional tensor-parallel sharding of the parameter.

4.4 Training Instrumentation: Capturing the Runtime Fields Rules Need

Once a rule is accepted, the detector must record the runtime state it reads: too little makes the rule unusable, too much adds unnecessary overhead. The Data Collector therefore lets the verified constraint library $\mathcal{P}$ determine which fields are required, rather than imposing one fixed trace schema: rules requiring only checksums and rank coordinates keep the collector at tier S0; rules needing optimizer state or control variables enable later tiers. Table 3 organizes S0–S6 as ordered schema tiers; the cost of raising a tier is evaluated in §5.5 and the full coverage–overhead curve is given in Appendix I (Figure I.1).

Table 3: Schema collection tiers used by the Data Collector. Each tier is opt-in; deployments choose the fields required by their active rule library.

Schema tier	Representative new fields
S0	cksum, param_name, ranks, step, stage, dtype, shape
S1	grad_norm, grad_cksum
S2	loss_value
S3	learning_rate, micro_step_id
S4	optimizer_state_cksum
S5–S6	ep/cp_rank, zero_stage, has_nan/inf, num_tokens

The collector instruments the PyTorch training loop at five canonical lifecycle points—`before-forward`, `after-forward`, `main-grad-in-backward`, `after-backward`, and `before-optimizer`—without modifying user code; three auxiliary taps (checkpoint save/load, `distributed.all_reduce`, build snapshot) extend coverage to the 8 hookpoints used by the verifier's indexer (§5.5), which include both optimizer pre- and post-step taps. The dominant overhead source is not the number of hooks but *what* each hook computes: full-tensor checksums require moving data to the CPU, so the collector uses GPU-side scalar reductions whenever a rule does not require bit-exact checksums. Captured records are written, with rank/step/stage coordinates unrolled into columns, into a DuckDB-backed trace database with a two-step sliding window—software-induced silent errors are deterministic, so cross-rank and cross-step checks need at most two consecutive snapshots. For the running example, the cross-rank replication rule requires only the S0 columns (`cksum`, `param_name`, `rank`, `step`, `stage`) at the `after-optimizer` (optimizer post-step) tap.

4.5 Online Detection: Compiling Verified Rules into Runtime Checks

After the required fields are written to the trace, the Verifier evaluates the accepted rules. Rather than evaluating each rule as a separate Python predicate with separate paths for state snapshots, call events, and initialization checks, TrainAudit compiles instantiated rules into parameterized SQL over the DuckDB trace: π_{topo} and π_{precond} become `WHERE` filters, and π_{schema} becomes the violation condition in the `HAVING` clause. An empty result means the constraint holds for the current step; a non-empty result *is* the violation set, already labeled with the offending parameter, step, and ranks.

Before the first training step, the verifier performs three tasks. First, it extracts the active topology τ from the launch script—DP/TP/PP/EP group sizes, `zero_stage`, and mixed-precision dtype—and disables rules whose π_{topo} cannot hold under it. Second, it builds a topology-aware lineage table that marks each parameter as replicated or intentionally sharded, so SQL filters remove sharded weights before π_{schema} is evaluated; this is the runtime counterpart of the SwitchMLP scope in §4.1. Third, it compiles every constraint into a parameterized SQL query. State-row, call-event, and initialization-snapshot rules differ only in which trace relations they read.

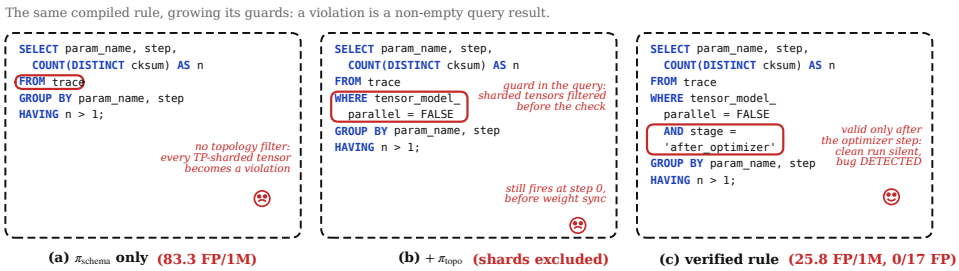


Figure 4: The compiled cross-rank replication rule, growing its guards. (a) With π_{schema} alone, every legitimately TP-sharded tensor is reported (83.3 FP/1M clean evaluations). (b) Adding π_{topo} filters sharded tensors inside the query, but the check still fires at step 0 before weight synchronization. (c) The verified rule carries both guards: it reduces the clean-trace false-positive rate to 25.8 FP/1M and produces 0/17 fixed-side false positives, while the SwitchMLP `router.weight` bug returns a non-empty result. A violation is a query result, not a post-hoc filter.

After each eligible event or scheduled snapshot, the Data Collector commits its records and the verifier executes the surviving rules bound to that hookpoint; event-triggered rules therefore run every step, whereas snapshot-bound rules follow the configured sampling period analyzed in §5.5. State-equality, call-ordering, and initialization violations all compile to the same query shape (full SQL specification in Appendix F).

When a query returns a non-empty row set, the verifier walks the corresponding rule's coarse-to-fine dependency tree to localize the fault: coarse rules (whole-tensor checksums) run first, and finer rules (per-module, per-stage, per-parameter, bitwise) instantiate only after a coarse check has confirmed that a fault is likely. The final report names the violating constraint, the affected parameter, the divergent ranks, and the step at which the violation first appeared. On the detected Real-SE cases, rule names alone localize 53% of cases, and the expand–contract drill-down compresses the remaining candidate sets by ~6×. Appendix G gives the full quantitative study.

5 EVALUATION

We evaluate whether (1) verified guarded constraints detect real silent errors that prior detectors miss, (2) each guard and verification stage is necessary, (3) the Pattern Catalog can be constructed reproducibly and covers held-out records, (4) the checks run at practical cost, and (5) the resulting library transfers across frameworks and versions. Real-SE replays run on one eight-NVIDIA-H200 node; collector microbenchmarks use a separate NVIDIA-H20 server with a 1.2B-parameter GPT harness.

5.1 Experimental Setup

We evaluate all detectors under a protocol that separates catalog construction from detector scoring and makes differences in their execution harnesses explicit.

Benchmark construction. Real-SE is the frozen, coverage-constrained benchmark used for detector evaluation, distinct from the 392-record evidence corpus used for catalog construction; it is not a random or prevalence-representative sample. It contains 17 detector-coverable native-commit method-level buggy/fixed replays and 1 predesignated observability-boundary case; Appendix B.5 details its construction, admission criteria, replay protocol, and coverage.

Baselines. We compare with two online detectors and two database-style alternatives. *Naïve Monitoring* checks loss and gradient norm for spikes and NaN/Inf values. *TrainCheck* [10], the closest prior system, infers behavioral invariants from execution traces using five relation templates; we use its official implementation. *Manual SQL* checks cross-rank checksum equality, numeric bounds, and counter monotonicity. *Daikon-style mining* [3] learns generic relation templates leave-one-configuration-out, without topology guards or counterexample verification.

Comparison and verdict protocol. TrainAudit and the two database-style baselines consume relational traces from the native-commit replays. TrainCheck instead runs on a small full-training surrogate aligned with each fault: it infers case-local invariants from the fixed surrogate and checks the buggy surrogate. Naïve Monitoring consumes the same surrogates' loss and gradient-norm series. This is a *case-aligned surrogate comparison*, not a same-harness comparison; Appendix I provides the per-case mappings and execution outcomes.

For TrainAudit, a true detection must flag the buggy replay, and any detector alert on its paired fixed-side execution is a false positive. The rule library is frozen before evaluation without access to case identifiers, fix commits, or verdicts. Because baseline fixed traces serve as inference/reference inputs rather than held-out checks, Table 4 leaves their fixed-side false-positive entries unreported. We report buggy-case detection, fixed-side false positives per case, and clean-trace false positives per million rule evaluations.

Platforms. The H200 replay node uses NVLink, PyTorch v2.1, and NCCL v2.18 unless a native revision requires otherwise; each framework case runs at its upstream buggy/fixed revisions rather than one global version.

5.2 Detection Effectiveness and Precision

Real-bug detection. As Table 4 shows, TrainAudit detects 17 of 18 cases: all 17/17 detector-coverable commit-pair cases and **94.4%** overall (95% Wilson CI [74.2%, 99.0%]), counting O-003 as a miss (per-case outcomes in Appendix I). Several cases share a framework

or rule, so the interval's independence assumption is optimistic. Because the Pattern Catalog and Real-SE come from the same historical pool, this number measures recall over that pool; Section 5.6 separately evaluates transfer and an out-of-pool discovery. Because executable commit pairs and compact drivers are selection requirements, Real-SE also over-represents bugs with localizable method-level witnesses; its detection rate must not be read as the prevalence of detectable errors in the full 392-record corpus.

Table 4: Real-SE detection. TrainAudit uses native-commit replays; baselines use the 17 completed case-aligned surrogates (O-003 pending). Baseline fixed-side checks are unavailable (—).

Method	Real-SE buggy detected	Fixed-side FP
TrainAudit (this work)	**17/18 (94.4%)**	**0/17 (0.0%)**
TrainCheck [10]	5/17 (29.4%)	—
Naïve Monitoring	0/17 (0.0%)	—

TrainCheck detects **5/17 (29.4%)** cases (11 misses and the O-005 inference failure), while Naïve Monitoring detects **0/17**. TrainCheck's single-process API relations capture sequential faults but cannot expose the cross-rank state, framework metadata, Python-layer counters, storage aliasing, and configuration-derived scopes needed by the remaining cases. None of the evaluated faults changes loss, gradient norm, or NaN/Inf within the surrogate window.

Database-oriented baselines. Table 5 asks whether standard constraints over the same trace relations suffice. Manual SQL expresses self-contained equality, bound, or monotonicity checks, while generic Daikon-style templates add learned single-trace relations; neither supplies the topology and phase guards needed to deploy the relations safely. On four clean Megatron-LM configurations, unguarded equality misclassifies 48% of multi-replica groups (4.8×10^5 FP/1M) because many tensors are intentionally sharded. Daikon-style templates, learned leave-one-configuration-out, retain 1.3×10^5 FP/1M, dominated by workload-specific ranges and equality over sharded tensors. Guarded, verified rules reduce this to 25.8 FP/1M.

Observability boundary. O-003 exhibits only sub-percent z-loss drift and changes no field that the online schema can stably distinguish without a trusted reference; it is therefore an observability limit rather than a missing rule. On the precision side, TrainAudit produces no false alert in all 17 paired fixed-side evaluations.[3] Thus the evaluated constraints catch every observable Real-SE case while avoiding indiscriminate alarms on their paired fixed-side outcomes.

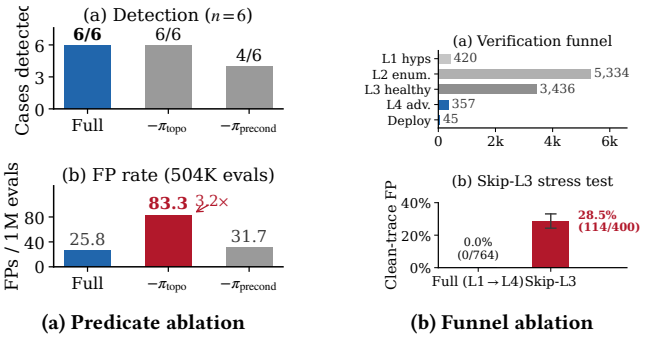
5.3 Impact of Guarded Verification

We isolate two decisions: whether runtime rules need both guards, and whether candidate constraints need healthy-run and adversarial verification. A complementary leave-one-database-out ablation

[3]Sixteen fixed sides complete cleanly; M-020's upstream fix instead rejects the faulty configuration as intended. We count this as a fixed-side verdict, not as a completed CLEAN replay.

Table 5: Database-style alternatives on the same trace relations: only guarded, verified rules reach single-digit clean FP/1M (four configurations).

	Manual SQL	Daikon-style	TrainAudit (verified)
Topology guard	no	no	**yes**
Verification	no	no	**yes**
Clean-trace FP/1M	4.8×10^5	1.3×10^5	**25.8**



(a) Predicate ablation **(b) Funnel ablation**

Figure 5: Guard and verification ablations. (a) Both runtime guards are load-bearing. (b) Funnel shrinkage (Table 6): detection stays flat while healthy-run and adversarial stages remove FP and non-detecting mass.

over 42 databases preserves the same ordering in every database: removing π_{precond}, adversarial verification, and π_{topo} raises aggregate false positives from 342 to 429, 551, and 598, respectively (funnel detail in Appendix E).

Predicate-level effect. We remove π_{topo} or π_{precond} on the six-case Real-SE multi-rank subset and 504 K clean rule evaluations (Figure 5a). Without π_{topo}, detection remains 6/6 but FP rises from 25.8 to 83.3 per million—the schema-only rule of Figure 4(a)—because replicated-state checks report intentionally sharded tensors. Without π_{precond}, premature step-0 equality alerts obscure two real violations, reducing detection to 4/6. The guarded configuration is the only one with 6/6 detection and fewer than 30 FP/1M.

Verification funnel. Across four frameworks, the automated funnel reduces 5,334 enumerated candidates to 3,436 healthy-run survivors and then 357 adversarially verified predicates (Figure 5b). Table 6 reports the dual axis that shrinkage alone cannot show: Real-SE detection versus clean-trace false positives. Each automated stage's library contains the next, so detection stays at **17/18 (94.4%)** from the L2 cohort through L4 and the curated deployment library—the sole residual miss is the observability-boundary case (§5.2). What changes is false-positive mass: skipping healthy-run validation (L2) yields 114/400 firings on held-out clean traces (28.5%, 95% Wilson CI [24.3%, 33.1%]), while L3 and L4 survivors produce 0/6,922 and 0/764 alerts on the same protocol. The adversarial stage then rejects 3,079 additional L3 survivors—98.6% workload-specific constants that a same-workload clean trace does not falsify—and that rejected set detects **0/18** Real-SE cases. Thus L3 removes clean-trace FP, while L4 removes non-detecting mass that clean FP no longer reveals; the final 45-rule deployment library is a separate, explicitly

Table 6: Funnel dual-axis check: contraction preserves Real-SE detection while removing false-positive and non-detecting mass. Detection uses the 18-case Real-SE protocol (§5.2). L2–L4 FP are held-out clean candidate–trace evaluations over independently rebuilt cohorts (within 2% of the mining counts; L2 subsampled); Deploy FP is fixed-side on paired Real-SE replays. Deploy is a curated subset of L4, not an automated stage.

Stage	|preds|	Real-SE det.	FP
L2 (skip healthy)	5,334	17/18 (94.4%)	114/400 clean
L3 (skip adversarial)	3,436	17/18 (94.4%)	0/6,922 clean
L4 (adversarial-pass)	357	17/18 (94.4%)	0/764 clean
Deploy (curated)	45	17/18 (94.4%)	0/17 fixed
L4 rejects	3,079	0/18 (0%)	—

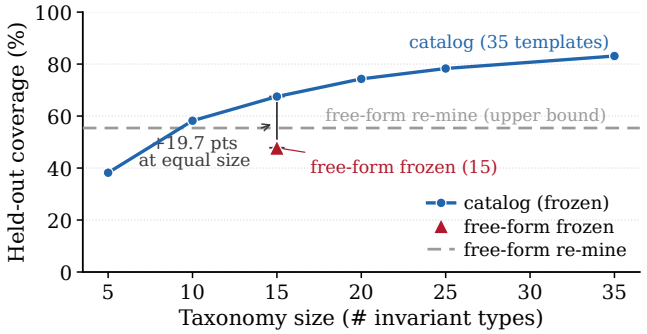


Figure 6: A frozen Pattern Catalog generalizes to held-out bugs through curation. Held-out coverage versus catalog size on the 249 later-added bugs. At 35 templates the frozen catalog covers 83.1%; at an equal budget of 15 templates it covers 67.5% versus 47.8% for a free-form catalog, and surpasses the 55.4% upper bound of a free-form model that re-mines every bug.

human-curated cross-framework subset of L4, not another automated gate, and preserves the same 17/18 (94.4%) with **0/17 (0.0%)** fixed-side false positives.

Together, the ablations show why both parts of the design are needed: runtime guards separate valid sharding and phases from violations, while the mining funnel prevents accidental clean-trace regularities from becoming deployed constraints.

5.4 Pattern-Catalog Reproducibility and Generalization

The Pattern Catalog earns its curation cost by transferring to bugs it was not built from. Holding out the 249 bugs added in the corpus's later expansion round, we ask for each whether any template in a candidate catalog would fire at runtime, so that the only variable is the catalog. As shown in Figure 6, the frozen 35-template catalog covers **95.1%** of the runtime-observable held-out bugs (83.1% overall) without ever seeing them, and at an equal budget of fifteen templates covers **67.5%** against **47.8%** for a free-form catalog—surpassing even the **55.4%** reached by a free-form model that re-mines every bug individually, and doing so at zero marginal cost

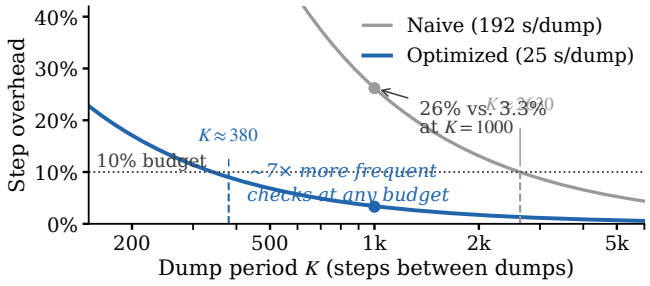


Figure 7: Amortized collection overhead vs. dump period K (1.2B/H20). The optimized collector meets any budget at a ~7× shorter period.

as the bug population grows. The advantage is a curation effect: 1,841 free-form proposals collapse into a handful of coarse buckets, whereas the catalog keeps thirty-five fine-grained obligations, each exercised on the holdout. Under the frozen locked-test protocol, two independent raters corroborate the automatic assignment: schema match **92.3%** (72/78), joint grounding **91.0%** (71/78), and template assignment agreement κ=0.954 (Appendix D).

5.5 Efficiency and Detection Latency

Rule-evaluation cost. Of the 45 curated cross-framework deployment rules (§5.3), the Megatron-LM evaluation snapshot activates 24 (12 A0 + 9 A1 + 3 probes) bound to 8 hookpoints; the remainder stay offline until grounded for that framework (Appendix I). Hookpoint indexing evaluates about three rules per tick rather than scanning all 24. Each compiled SQL query takes 2–3 ms, dominated by DuckDB planning, giving an observed upper bound of roughly 10 ms per tick on the H20 harness.

Snapshot collection. We separately microbenchmark snapshot collection on the 1.2B/H20 harness (DP=2, bf16). Against a 732 ms uninstrumented step, a full 13,106-row snapshot using per-parameter SHA-256 and host statistics takes 192 s. Replacing synchronous GPU-to-CPU hashing with a deterministic on-device fingerprint cuts this to 27.5 s; GPU-side statistics reduce it further to 25 s, a 7.7× improvement (Table 7). The same mechanism shapes the schema tiers of §4.4: because the cost is dominated by the synchronous copy rather than by the number of fields, raising the tier from checksums to GPU-side gradient statistics lowers per-step overhead from ~8% to ~1.5% while widening the observable surface (Appendix I, Figure I.1).

Table 7: Snapshot microbenchmark at 1.2B/H20 (baseline: 732 ms/step): GPU fingerprinting cuts a full dump from 192 s to 25 s.

Collector configuration	dump/step	vs. baseline
Naive (SHA-256 + host stats), full	192 s	262×
+ GPU fingerprint (checksum)	27.5 s	38×
+ fused GPU statistics	**25 s**	**34×**

Overhead–latency trade-off. The optimized and SHA-256 paths produce identical cross-rank equality verdicts; only their implementation differs. From the measured 25 s dump cost, amortizing

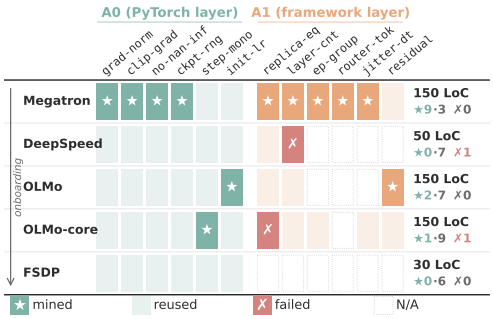


Figure 8: Grounded-rule provenance across five adapters under one catalog version: new mining falls from nine rules to zero by FSDP. ★ = first grounding; shade = executable rule reused unchanged; × = template retained but topology scope needs re-grounding; dash = no corresponding hookpoint.

a snapshot every K steps yields a *derived* overhead below 10% at $K \approx 380$ and below 5% at $K \approx 760$—about 4.6 and 9.3 minutes at the measured step time. As Figure 7 shows, the naive collector requires $K \approx 2630$ for the same 10% budget, so the optimized collector checks ~7× more frequently at any overhead target. Event-triggered rules fire on the first recorded event; snapshot-bound rules have a worst-case delay of K steps. Across the measured TP/PP/DP variants, optimized dump cost ranges from 15 to 51 s, so K must be set per topology rather than inferred from world size.

Offline cost. Mining Megatron-LM's deployed rules takes roughly 350 LLM iterations and 40–50 M tokens (about $70–100 at the evaluated model price); this cost is paid offline and only when the framework library is regenerated.

5.6 Portability and Production Experience

Adapter portability. We deploy grounded rules from the same catalog version through adapters for Megatron-LM, DeepSpeed, OLMo, OLMo-core, and FSDP (Figure 8). The provenance matrix distinguishes direct rule reuse from template reuse with target-specific re-grounding. New rule mining decreases from nine rules for the first framework to none for FSDP, which directly inherits all six framework-agnostic A0 rules through a 30-line adapter. Two A1 rules cannot migrate unchanged because PP partition granularity and replica grouping alter π_{topo}; their catalog templates remain applicable, but their target-framework topology scopes must be re-grounded and verified.

Transfer analysis. Treating Megatron-LM and DeepSpeed as sources, their rules detect 8/10 OLMo/OLMo-core cases—8/8 observable under the schema—with no case-level false positive; seven detections use A0 rules. This is an upper bound, because the deployed library was assembled after all four frameworks were onboarded, not a from-scratch source-only re-mine. At the catalog level, every detector-observable Real-SE case matches a template in the evaluated catalog snapshot; the rule-level shortfall therefore lies in observability or framework grounding rather than a second template vocabulary. At the input level, none of 150 archived mining logs contains a Real-SE fix commit or issue/PR text. These controls reduce direct case leakage but do not make Real-SE an out-of-pool benchmark. A0 rules also remain unchanged across five Megatron-LM commits spanning 23 months.

Production experience. On a presumed-clean 256×H200 production job with an MTP-only pipeline stage, the pinned deployment version's param-group rule fired at initialization, before the first optimizer step: tied `word_embeddings` replicas were routed to different optimizers across PP ranks. Loss, gradient norm, throughput, and checkpoint operations remained normal. The report identified the missing embedding attribute that routed one replica to Muon and another to AdamW. Had the planned one-week job continued, it would have consumed approximately 43,000 H200 GPU-hours with divergent optimizer state. We reported the fault upstream, where maintainers confirmed and fixed the previously unrecorded bug; the same rule reproduces it on a Megatron-LM HEAD scan. This is one out-of-pool discovery intercepted in production, not two independent bugs.

Limitations. Overhead is measured on one node, so multi-node collection cost remains unquantified. Clean-run FP measurements cover finite 200-step windows, and the 256×H200 result is one initialization-time interception rather than a long-horizon deployment study. Finally, the provenance audit excludes case material from mining inputs but cannot exclude public bug discussions from LLM pretraining.

6 RELATED WORK

Prior silent-error detectors differ in *oracle source* (Table 1). Outcome-metric monitors [15, 16, 18, 26] fire only after macro curves bend, catching none of the 17 completed case-aligned surrogate cases (§5.2). TrainCheck [10], the closest online prior work, infers behavioral invariants from single-process API traces via five relation templates and reaches **5/17** on those surrogates; it cannot express cross-rank state or topology-conditioned guards. TTrace [9] and TrainVerify [14] need an alignable reference or a static plan and therefore run offline or before the first step. TrainAudit instead derives deployable checks from framework source and historical bug evidence, attaining online cross-rank coverage without a trusted reference. Bug studies such as TaxDC [6, 12, 27] inform corpus coding; ours provides a large-scale coded corpus of upstream silent-error evidence in distributed LLM training and induces automatically applied relation templates from it. Hybrid-parallel frameworks [8, 23] and distributed training systems [13, 20, 21] widen the fragility surface these checks defend.

A long line of data-management research expresses correctness as constraints over relations: conditional functional dependencies [4] attach value conditions to functional dependencies, and denial constraints [1] generalize these to arbitrary tuple-pair predicates. Production validators—Auto-Validate [25] and Deequ [22]—operationalize this view at scale but validate *static* columns. TrainAudit inherits the relational-constraint view but binds each obligation to the live training trace: π_{schema} selects measurable fields, while π_{topo} and π_{precond} supply the topology and phase guards these formalisms lack, lifting data-quality constraints from static tables to online distributed jobs. On the same trace relations, hand-written SQL and Daikon-style mining [3] produce 4.8×10^5 and 1.3×10^5 false positives per million rule evaluations without guards or verification (§5.2, Table 5).

QURE [24] verifies LLM-translated SQL via counterexample-based checking, a pattern our mining funnel adopts (§4.3); we extend that convergence to *training execution*.

LLMs can propose invariants from programs [5, 19] and infer API specifications [17], but target sequential programs and single-library contracts; classic miners such as Daikon [3] lack topology and phase guards and over-fit workload coincidences (§5.3). TrainAudit is, to our knowledge, the first to mine *verified guarded relational constraints* for silent-error detection in distributed LLM training.

7 CONCLUSION

TrainAudit formulates silent-error detection as checking verified, topology- and phase-guarded relational constraints over live training traces. On Real-SE, it detects **17/18 (94.4%)** buggy instances with **0/17** fixed-side false positives, outperforming existing online detectors on the completed case-aligned surrogates. In a 256×H200 production job, it identified a previously unrecorded bug before the first optimizer step, preventing approximately 43,000 GPU-hours of silent corruption. Although detection remains bounded by the instrumented trace, our results show that many silent errors are not signal-free: their evidence becomes actionable when distributed-training semantics are encoded as verified guarded relations.

REFERENCES

[1] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering Denial Constraints. *Proceedings of the VLDB Endowment (PVLDB)* 6, 13 (2013), 1498–1509. https://doi.org/10.14778/2536258.2536262

[2] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. 2021. Silent Data Corruptions at Scale. arXiv:2102.11245 [cs.AR] https://arxiv.org/abs/2102.11245

[3] Michael D Ernst, Jeff H Perkins, Philip J Guo, Stephen McCamant, Carlos Pacheco, Matthew S Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *Science of computer programming* 69, 1-3 (2007), 35–45. https://doi.org/10.1016/j.scico.2007.01.015

[4] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional Functional Dependencies for Capturing Data Inconsistencies. *ACM Transactions on Database Systems (TODS)* 33, 2 (2008), 6:1–6:48. https://doi.org/10.1145/1366102.1366103

[5] Stewart Grant, Hendrik Cech, and Ivan Beschastnikh. 2018. Inferring and asserting distributed system invariants. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg, Sweden, 1149–1159. https://doi.org/10.1145/3180155.3180199

[6] Haryadi S Gunawi, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tiratat Patana-anake, Thanh Do, Jeffry Adityatama, Shan Lu, et al. 2014. What Bugs Live in the Cloud? A Study of 3000+ Issues in Cloud Systems. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. ACM, Seattle, WA, USA, 1–14. https://doi.org/10.1145/2670979.2670986

[7] Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. 2021. Cores That Don't Count. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*. ACM, Ann Arbor, MI, USA, 9–16. https://doi.org/10.1145/3458336.3465297

[8] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. *Advances in neural information processing systems* 32 (2019), 103–112. https://papers.nips.cc/paper/8717-gpipe-efficient-training-of-giant-neural-networks-using-pipeline-parallelism

[9] Haitian Jiang, Shaowei Zhu, Zhen Zhang, Zhenyu Song, Xinwei Fu, Zhen Jia, Yida Wang, and Jinyang Li. 2025. TTrace: Lightweight Error Checking and Diagnosis for Distributed Training. arXiv:2506.09280 [cs.DC] https://arxiv.org/abs/2506.09280

[10] Yuxuan Jiang, Ziming Zhou, Boyu Xu, Beijie Liu, Runhui Xu, and Peng Huang. 2025. Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association, Boston, MA, USA, 313–329. https://www.usenix.org/conference/osdi25/presentation/jiang

[11] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*.

[12] Tanakorn Leesatapornwongsa, Jeffrey F Lukman, Shan Lu, and Haryadi S Gunawi. 2016. TaxDC: A Taxonomy of Non-Deterministic Concurrency Bugs in Datacenter Distributed Systems. In *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, Atlanta, GA, USA, 517–530. https://doi.org/10.1145/2872362.2872374

[13] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. PyTorch distributed: experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3005–3018. https://doi.org/10.14778/3415478.3415530

[14] Yunchi Lu, Youshan Miao, Cheng Tan, Peng Huang, Yi Zhu, Xian Zhang, and Fan Yang. 2025. TrainVerify: Equivalence-Based Verification for Distributed LLM Training. arXiv:2506.15961 [cs.LG]

[15] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S Vetter. 2018. NVIDIA Tensor Core Programmability, Performance & Precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, Vancouver, BC, Canada, 522–531. https://doi.org/10.1109/IPDPSW.2018.00091

[16] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2018. Mixed Precision Training. In *International Conference on Learning Representations*. OpenReview.net, Vancouver, BC, Canada, 10. https://openreview.net/forum?id=r1gs9JgRZ

[17] Rahul Pandita, Xusheng Xiao, Hao Zhong, Tao Xie, Stephen Oney, and Amit Paradkar. 2012. Inferring Method Specifications from Natural Language API Descriptions. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. IEEE, Zurich, Switzerland, 815–825. https://doi.org/10.1109/ICSE.2012.6227137

[18] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 28. PMLR, Atlanta, GA, USA, 1310–1318.

[19] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 202. PMLR, Honolulu, HI, USA, 27496–27520. https://proceedings.mlr.press/v202/pei23a.html

[20] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, Atlanta, GA, USA, 1–16. https://doi.org/10.1109/SC41405.2020.00024

[21] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. ACM, Virtual Event, 3505–3506. https://doi.org/10.1145/3394486.3406703

[22] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. 2018. Automating Large-Scale Data Quality Verification. *Proceedings of the VLDB Endowment (PVLDB)* 11, 12 (2018), 1781–1794. https://doi.org/10.14778/3229863.3229867

[23] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL] https://arxiv.org/abs/1909.08053

[24] Tarique Siddiqui, Arnd Christian König, Jiashen Cao, Cong Yan, and Shuvendu K. Lahiri. 2025. QURE: AI-Assisted and Automatically Verified UDF Inlining. *Proc. ACM Manag. Data* 3, 1, Article 66 (feb 2025), 26 pages. https://doi.org/10.1145/3709716

[25] Jie Song and Yeye He. 2021. Auto-Validate: Unsupervised Data Validation Using Data-Domain Patterns Inferred from Data Lakes. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, Xi'an, China, 1678–1691. https://doi.org/10.1145/3448016.3457250

[26] Naigang Wang, Jungwook Choi, Daniel Brand, Chia-Yu Chen, and Kailash Gopalakrishnan. 2018. Training deep neural networks with 8-bit floating point numbers. *Advances in neural information processing systems* 31 (2018), 7675–7684. https://papers.nips.cc/paper/7997-training-deep-neural-networks-with-8-bit-floating-point-numbers

[27] Yuhao Zhang, Yifan Chen, Shing-Chi Cheung, Yingfei Xiong, and Lu Zhang. 2018. An Empirical Study on TensorFlow Program Bugs. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Amsterdam, Netherlands, 129–140. https://doi.org/10.1145/3213846.3213866